

STM32CubeIDE Debugging Session Summary

1 Pinout and System View Discussion

In our STM32CubeIDE session, we started by discussing the pinout and system views in the `.ioc` file. Some key points from this phase:

- Pins we looked at included:
 - PC-13: B1 Blue push button
 - PA-5: LD2 (green LED)
 - PA-2 and PA-3: USART_TX and USART_RX, respectively
 - RCC-related pins like PC-14, PC-15, PH0, and PH1
- We explored the **System View**, where we noticed:
 - A green checkmark next to most items in the **System Core** section.
 - NVIC (Interrupt Controller) and the interrupt table indicating which interrupts are enabled.
 - Warnings associated with the SWO and other peripherals, signaling configuration issues or unconnected pins.

2 Debugging Session: From Reset_Handler to main()

We stepped through the debugging process from the `Reset_Handler`, observing key steps involved in initialization before reaching `main()`. Here are the details:

2.1 Reset_Handler Initialization

- `Reset_Handler` starts by loading the `_estack` into the stack pointer (SP). This defines the initial stack location.
- After that, we branched to `SystemInit`, which handles basic system configurations such as setting the clock tree and enabling the floating-point unit (FPU).
- The instruction we examined in detail: `ldr r5, [pc, #52]` loads an address relative to the program counter (PC) into register `r5`, accounting for ARM's two-instruction prefetch in Thumb mode.

2.2 SystemInit and FPU Initialization

- We encountered a block of code to enable the FPU:

```
#if (__FPU_PRESENT == 1) && (__FPU_USED == 1)
SCB->CPACR |= ((3UL << 10*2)|(3UL << 11*2)); /* Set CP10 and CP11 Full Access */
#endif
```
- The `SCB->CPACR` register (0xE000ED88) controls access to the FPU, and we noted that this masked the two bits associated with CP10 and CP11, setting them to 0x3.
- We confirmed in the debugger that these two-bit fields were modified after stepping over this line of code.

2.3 Loading Data into SRAM

- The `Reset_Handler` proceeds by initializing the data segment, copying it from flash into SRAM using global markers like `_sdata`, `_edata`, and `_sidata` defined in the linker file.
- We discussed how these global symbols are used to copy initialized data from flash (non-volatile) to SRAM (volatile), which allows the program to modify its variables in memory during runtime.

2.4 `__libc_init_array`: Static Constructors

- We stepped into the `__libc_init_array` function, which is responsible for calling static constructors in C++ programs before `main()` is reached.
- The hardware automatically handles loading the `init_array` addresses from memory, and we examined how `ldr` instructions loaded the constructor pointers for the initialization process.

2.5 Reaching `main()`

- After the data segment was loaded into SRAM and static constructors initialized, we hit `main()`.
- We paused here to reflect on the fact that while we had only reached `main()`, we had already covered a significant amount of low-level initialization logic that directly correlates with operating system concepts like memory management, interrupt handling, and context switching.

3 Key Observations

- Debugging through the startup sequence provided valuable insight into the microcontroller's hardware-level setup, reinforcing concepts from operating systems and systems programming.
- The process of stepping through the `Reset_Handler` and observing how the microcontroller handles memory, data segments, and FPU initialization offers a deeper understanding of embedded systems architecture.
- We confirmed that ARM's PC-relative addressing requires awareness of the prefetch mechanism, and we saw firsthand how global symbols from the linker file drive the initialization process.